

Anexo 09 — Motor de animacion integrado (producto final)

Archivo: Scripts/run_animation_engine.py

Para que sirvio:

Ejecutable final. Combina la policy de cabeza y la del brazo en una sola simulacion, con maquina de estados (reposo / agarrar / saludar), autofocus procedural, camara orbital controlable, esquemas de color cambiables y el servidor web. Es lo que se corre para operar el robot.

```
1 | """Motor de animacion DUM4 – runtime integrado.
2 |
3 | Combina:
4 | - v13 head policy: trackea el target activo (red ball o yellow ball segun estado)
5 | - v12/v13 arm policy: realiza grab + hold + throw cuando se dispara
6 | - Web server: slider para mover el red mocap + boton "agarrar bola amarilla"
7 | - State machine simple: IDLE <-> GRAB_CYCLE
8 |
9 | Flow:
10 |     IDLE (default):
11 |         - red ball mocap movido por slider web
12 |         - head policy: target = red ball pos
13 |         - arm: ctrl neutro (no policy)
14 |         - yellow ball: oculta (z = -10)
15 |
16 |     GRAB_CYCLE (al apretar boton):
17 |         1. spawn yellow_ball arriba de la palma (palm_z + 0.40)
18 |         2. head policy: target = yellow_ball pos
19 |         3. arm policy: activa
20 |         4. env state machine (interno): FALLING -> HELD -> THROWN
21 |         5. en THROWN, aplicar xfrc para reducir gravedad efectiva ~80%
22 |         -> bola flota 2-3 segundos
23 |         6. al touchdown o timeout 3s post-release: volver a IDLE
24 |         7. head policy vuelve a target = red ball
25 |
26 | Uso:
27 |     python Scripts/run_animation_engine.py
28 |     [--arm-policy runs/grab_phasel_v13_natural/final.zip]
29 |     [--head-policy runs/ppo_dum_v13/final.zip]
30 |     [--host 0.0.0.0 --port 8000]
31 |
32 | Web: http://localhost:8000
33 | """
34 | from __future__ import annotations
35 |
36 | import argparse
37 | import io
38 | import queue
39 | import sys
40 | import threading
41 | import time
42 | from pathlib import Path
43 |
44 | SCRIPTS_DIR = Path(__file__).resolve().parent
45 | sys.path.insert(0, str(SCRIPTS_DIR))
46 |
47 | import numpy as np
48 | import mujoco
49 | import mujoco.viewer
50 | from PIL import Image
51 | from stable_baselines3 import PPO
52 | from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
53 |
54 | from rl_env import DUMHeadTrackingEnv
55 | from rl.envs.grab_env import (
56 |     DUMGrabEnv, PALM_REST_WORLD, INIT_BALL_ABOVE_PALM,
57 |     THROW_HELD_MAX_S, B_GRAB_SIMPLE, THROW_GRAB_BONUS,
58 | )
59 | from rl.procedural.wave import WaveAnimation
60 | from skins import apply_skin, DEFAULT_SKIN
61 | import web_remote.server as server
62 |
63 |
64 | # ----- Estados del motor -----
65 |
66 | STATE_IDLE = "IDLE"
67 | STATE_SPAWN_AND_FALL = "SPAWN_AND_FALL"
68 | STATE_HELD = "HELD"
69 | STATE_THROWN = "THROWN"
70 | STATE_FOLLOW = "FOLLOW" # 3s post-release siguiendo la bola con cabeza
71 | STATE_WAVING = "WAVING" # saludo procedural (4s)
72 |
73 |
```

```

74 | # Autofoco animatronico (replica del DUMHeadTrackingEnv en rl_env.py)
75 | FOCUS_THRESHOLD = 0.07          # rad ≈ 4° – bajo de esto se dispara el foco
76 | FOCUS_SETTLING_STEPS = 10
77 | FOCUS_ANIM_STEPS = 50          # ~1s a 50Hz
78 | FOCUS_REARM_LOST_STEPS = 30    # 0.6s out-of-focus para rearmar
79 | FOCUS_REARM_THRESHOLD_MULT = 3.0
80 | FOCUS_N_CYCLES_RANGE = (1, 4)
81 | FOCUS_AMPLITUDE_RANGE = (0.5, 1.0)
82 | FOCUS_RESTING_RANGE = (0.0, 0.15)
83 | HEAD_FORWARD_LOCAL = np.array([0.0, -1.0, 0.0])
84 |
85 |
86 | # ----- Constantes -----
87 |
88 | YELLOW_BALL_HIDDEN_Z = -10.0   # cuando no se usa, la mandamos lejos
89 | GRAB_CYCLE_SPAWN_ABOVE = 0.40  # spawn yellow_ball a palm + 0.40m
90 | FOLLOW_DURATION_S = 3.0        # segundos post-throw siguiendo la bola
91 | GRAVITY_CANCEL_FACTOR = 0.80   # durante THROWN, cancelamos 80% de la gravedad
92 | SIM_HZ = 50
93 | FRAME_SKIP = 4
94 | TIMESTEP = 0.005
95 |
96 | # NO MAS THROW BOOST: la velocidad del throw sale 100% de la policy.
97 | # Si la bola no llega lejos, mas training (v14+).
98 |
99 |
100 | def main():
101 |     p = argparse.ArgumentParser()
102 |     p.add_argument("--arm-policy", type=str,
103 |                   default="runs/grab_phase1_v16_extended/final.zip",
104 |                   help="Arm policy zip. Default v16_extended (cap 35/15). "
105 |                       "Si querés volver al v14_long (mejor catch global, cap 40/10), "
106 |                       "pasar --arm-policy runs/grab_phase1_v14_long/final.zip")
107 |     p.add_argument("--head-policy", type=str,
108 |                   default="runs/ppo_dum_v14c_aggressive/final.zip",
109 |                   help="Head policy zip. Default v14c (usa HeadRot al limite fisico ±160°). "
110 |                       "v13 original tenia cono ±75° y compensaba con HeadBase tilt.")
111 |     p.add_argument("--host", type=str, default="0.0.0.0")
112 |     p.add_argument("--port", type=int, default=8000)
113 |     p.add_argument("--no-viewer", action="store_true",
114 |                   help="No abrir viewer interactivo (util para headless con solo web stream)")
115 |     args = p.parse_args()
116 |
117 |     # Fallback chain: v16 -> v14_long -> v13 -> error
118 |     arm_path = Path(args.arm_policy)
119 |     if not arm_path.exists():
120 |         fallback_chain = [
121 |             SCRIPTS_DIR.parent / "runs" / "grab_phase1_v14_long" / "final.zip",
122 |             SCRIPTS_DIR.parent / "runs" / "grab_phase1_v13_natural" / "final.zip",
123 |         ]
124 |         for fallback in fallback_chain:
125 |             if fallback.exists():
126 |                 arm_path = fallback
127 |                 print(f"[setup] arm policy default no existe, fallback a {arm_path.parent.name}")
128 |                 break
129 |         else:
130 |             raise FileNotFoundError(f"No arm policy: {args.arm_policy} ni fallbacks")
131 |     print(f"[setup] arm policy: {arm_path}")
132 |
133 |     # --- Cargar arm policy + VecNormalize ---
134 |     dummy_arm = DummyVecEnv([lambda: DUMGrabEnv(phase=1)])
135 |     arm_vn_path = arm_path.parent / "vecnormalize.pkl"
136 |     arm_vn = VecNormalize.load(str(arm_vn_path), dummy_arm)
137 |     arm_vn.training = False
138 |     arm_vn.norm_reward = False
139 |     arm_model = PPO.load(str(arm_path), env=arm_vn)
140 |     arm_env = arm_vn.venv.envs[0]
141 |     # Activar throw permanentemente (el state machine externo controla cuando se "usa")
142 |     arm_env.set_throw_enabled(True)
143 |     arm_env.set_falling_active(True)
144 |     arm_env.set_subphase(0)
145 |
146 |     # --- Cargar head policy (v13 sin VecNormalize) ---
147 |     head_model = PPO.load(args.head_policy)
148 |     print(f"[setup] head policy: {args.head_policy}")
149 |
150 |     model = arm_env.model
151 |     data = arm_env.data
152 |
153 |     # Skin inicial del robot (cambiable en runtime desde la web)
154 |     apply_skin(model, DEFAULT_SKIN)
155 |     print(f"[setup] skin inicial: {DEFAULT_SKIN}")
156 |
157 |     # --- IDs adicionales para el head obs ---
158 |     head_body = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, "LenteExt_link")

```

```

159 | lens_site = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, "lens_center")
160 | head_joint_names = ["Neck_joint", "HeadBase_joint", "HeadRotation_joint"]
161 | head_joint_ids = [mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, n) for n in head_joint_names]
162 | head_qpos_adr = np.array([model.jnt_qposadr[j] for j in head_joint_ids])
163 | head_dof_adr = np.array([model.jnt_dofadr[j] for j in head_joint_ids])
164 | hb_jid = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, "HipBody_joint")
165 | hb_qpos_adr = model.jnt_qposadr[hb_jid]
166 | hb_dof_adr = model.jnt_dofadr[hb_jid]
167 | head_act_names = ["act_Neck", "act_HeadBase", "act_HeadRot"]
168 | head_act_ids = [mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_ACTUATOR, n) for n in head_act_names]
169 | head_ctrl_lo = np.array([model.actuator_ctrlrange[a, 0] for a in head_act_ids])
170 | head_ctrl_hi = np.array([model.actuator_ctrlrange[a, 1] for a in head_act_ids])
171 | # Lente actuator para autofocus
172 | lens_act_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_ACTUATOR, "act_LenteExt")
173 | lens_ctrl_lo, lens_ctrl_hi = model.actuator_ctrlrange[lens_act_id]
174 | LENS_MAX = float(lens_ctrl_hi) # ~0.021
175 | # Red ball mocap (target visual del slider web)
176 | target_mocap_body = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, "target")
177 | target_mocap_idx = model.body_mocapid[target_mocap_body]
178 |
179 | # --- Colas y servidor web ---
180 | target_q: queue.Queue = queue.Queue(maxsize=1)
181 | telemetry_q: queue.Queue = queue.Queue(maxsize=1)
182 | frame_q: queue.Queue = queue.Queue(maxsize=1)
183 | event_q: queue.Queue = queue.Queue(maxsize=8)
184 | server.init_queues(target_q, telemetry_q, frame_q, event_q)
185 |
186 | def run_uvicorn():
187 |     import uvicorn
188 |     config = uvicorn.Config(server.app, host=args.host, port=args.port, log_level="warning")
189 |     srv = uvicorn.Server(config)
190 |     srv.run()
191 |
192 | th = threading.Thread(target=run_uvicorn, daemon=True, name="uvicorn")
193 | th.start()
194 | print(f"[server] http://{args.host}:{args.port}")
195 | print(f"[server] local: http://127.0.0.1:{args.port}")
196 |
197 | # --- Estado del runtime ---
198 | sm_state = STATE_IDLE
199 | red_target_world = np.array([-0.012, -0.50, 0.50]) # default red mocap pos
200 | yellow_hidden_pos = np.array([0.0, 0.0, YELLOW_BALL_HIDDEN_Z])
201 | target_world_prev = red_target_world.copy()
202 | prev_head_action = np.zeros(3, dtype=np.float32)
203 | cycle_start_t = 0.0
204 | follow_start_t = None
205 | grav_default = float(arm_env._current_falling_gravity)
206 |
207 | # --- Estado del autofocus (replica de DUMHeadTrackingEnv) ---
208 | focus_triggered = False
209 | focus_anim_step = -1
210 | focus_amplitude = 0.7
211 | focus_n_cycles = 2
212 | focus_resting = 0.0
213 | focus_lost_steps = 0
214 | focus_settling_counter = 0 # cuenta steps con head settled (para SETTLING_STEPS)
215 |
216 | # --- Wave (saludo) ---
217 | wave_left = WaveAnimation(side="left", model=model)
218 | wave_right = WaveAnimation(side="right", model=model)
219 | wave_current = None # apunta a wave_left o wave_right cuando activo
220 | wave_start_t = None
221 |
222 | # Renderer offscreen para stream MJPEG. Usamos directamente 480x480 (lo que el
223 | # framebuffer del modelo soporta). CSS object-fit:cover llena el panel.
224 | stream_renderer = mujoco.Renderer(model, height=480, width=480)
225 | print(f"[setup] renderer 480x480 (CSS object-fit:cover lo escala al panel)")
226 | frame_every_n = 2
227 | frame_counter = 0
228 |
229 | # --- Camara orbital del stream (D-pad desde la web) ---
230 | # Siempre centrada en el robot (lookat fijo) a distancia fija -> orbita en una
231 | # esfera. Izq/der giran (azimuth); arriba/abajo cambian la ALTURA (elevation),
232 | # con un piso: la camara no baja de la altura actual (solo puede subir).
233 | CAM_LOOKAT = (0.07, 0.07, 0.32)
234 | CAM_DISTANCE = 1.5
235 | CAM_AZ_STEP = 30.0 # grados por click izq/der
236 | CAM_EL_STEP = 15.0 # grados por click arriba/abajo
237 | CAM_EL_LOW = -12.0 # altura MINIMA (la actual) - la camara no baja de aca
238 | CAM_EL_HIGH = -85.0 # altura MAXIMA (casi cenital)
239 | stream_cam = mujoco.MjvCamera()
240 | stream_cam.type = mujoco.mjtCamera.mjCAMERA_FREE
241 | stream_cam.lookat[:] = CAM_LOOKAT
242 | stream_cam.distance = CAM_DISTANCE
243 | stream_cam.azimuth = 90.0 # frente del robot

```

```

244 | stream_cam.elevation = CAM_EL_LOW
245 |
246 | # Hide yellow ball initially
247 | arm_env.data.qpos[arm_env._ball_qpos_adr:arm_env._ball_qpos_adr+3] = yellow_hidden_pos
248 | arm_env.data.qvel[arm_env._ball_dof_adr:arm_env._ball_dof_adr+6] = 0.0
249 | mujoco.mj_forward(model, data)
250 |
251 | def hide_yellow_ball():
252 |     data.qpos[arm_env._ball_qpos_adr:arm_env._ball_qpos_adr+3] = yellow_hidden_pos
253 |     data.qvel[arm_env._ball_dof_adr:arm_env._ball_dof_adr+6] = 0.0
254 |
255 | def spawn_yellow_ball():
256 |     # Aleatorizar side (R o L)
257 |     arm_env._side = "R" if np.random.uniform() < 0.5 else "L"
258 |     # Randomizar color
259 |     if arm_env._ball_geom_id >= 0:
260 |         rgb = np.random.uniform(0.3, 1.0, size=3)
261 |         arm_env.model.geom_rgba[arm_env._ball_geom_id, :3] = rgb
262 |         arm_env.model.geom_rgba[arm_env._ball_geom_id, 3] = 1.0
263 |     # Spawn arriba de la palma activa
264 |     ee = data.site_xpos[arm_env._ee_site_id]
265 |     spawn = np.array([float(ee[0]), float(ee[1]), float(ee[2]) + GRAB_CYCLE_SPAWN_ABOVE])
266 |     data.qpos[arm_env._ball_qpos_adr:arm_env._ball_qpos_adr+3] = spawn
267 |     data.qpos[arm_env._ball_qpos_adr+3:arm_env._ball_qpos_adr+7] = [1.0, 0.0, 0.0, 0.0]
268 |     data.qvel[arm_env._ball_dof_adr:arm_env._ball_dof_adr+6] = 0.0
269 |     # Reset state del env interno
270 |     from rl.envs.grab_env import STATE_FALLING
271 |     arm_env.state = STATE_FALLING
272 |     arm_env._grab_bonus_consumed = False
273 |     arm_env.t_grab = None
274 |     arm_env.t_release = None
275 |     arm_env.was_held = False
276 |     arm_env._sim_t = 0.0
277 |     mujoco.mj_forward(model, data)
278 |
279 | def compute_head_obs(target_world, prev_action):
280 |     head_mat = data.xmat[head_body].reshape(3, 3)
281 |     lens_origin = data.site_xpos[lens_site]
282 |     tloc = head_mat.T @ (target_world - lens_origin)
283 |     tdist = float(np.linalg.norm(tloc))
284 |     tdir = tloc / (tdist + 1e-9)
285 |     dt_ctrl = FRAME_SKIP * TIMESTEP
286 |     tvel_world = (target_world - target_world_prev) / dt_ctrl
287 |     tvel = head_mat.T @ tvel_world
288 |     qpos_head = data.qpos[head_qpos_adr]
289 |     qvel_head = data.qvel[head_dof_adr]
290 |     qpos_hb = float(data.qpos[hb_qpos_adr])
291 |     qvel_hb = float(data.qvel[hb_dof_adr])
292 |     return np.concatenate([
293 |         qpos_head, qvel_head, [qpos_hb, qvel_hb],
294 |         tdir, [tdist], tvel,
295 |         prev_action.astype(np.float64),
296 |     ])
297 |
298 | def head_action_to_ctrl(action):
299 |     a = np.clip(action, -1.0, 1.0).astype(np.float64)
300 |     return (a + 1.0) * 0.5 * (head_ctrl_hi - head_ctrl_lo) + head_ctrl_lo
301 |
302 | print(f"[runtime] estado inicial: {sm_state}")
303 | print(f"[runtime] CTRL+C para terminar")
304 |
305 | sim_dt = FRAME_SKIP * TIMESTEP # 0.02s = 50 Hz de control
306 | viewer_ctx = None
307 | if not args.no_viewer:
308 |     viewer_ctx = mujoco.viewer.launch_passive(model, data)
309 |     print(f"[viewer] abierto. Cerrar el viewer termina el programa.")
310 |
311 | last_log_t = time.perf_counter()
312 | sim_t_total = 0.0
313 | last_telemetry_t = 0.0
314 |
315 | try:
316 |     while True:
317 |         loop_start = time.perf_counter()
318 |         if viewer_ctx is not None and not viewer_ctx.is_running():
319 |             break
320 |
321 |         # --- 1. Leer eventos del web (botones "agarrar", "saludar") ---
322 |         try:
323 |             ev = event_q.get_nowait()
324 |             ev_type = ev.get("type")
325 |             if ev_type == "grab_yellow" and sm_state == STATE_IDLE:
326 |                 print(f"[runtime] EVENT grab_yellow recibido -> SPAWN_AND_FALL")
327 |                 spawn_yellow_ball()
328 |                 # Reset target_world_prev a la NUEVA posicion de la bola para

```

```

329 |         # evitar tvel_local explosivo (el bola "salto" de z=-10 a palm_z).
330 |         new_ball_pos = data.qpos[arm_env._ball_qpos_adr:arm_env._ball_qpos_adr+3].copy()
331 |         target_world_prev = new_ball_pos
332 |         sm_state = STATE_SPAWN_AND_FALL
333 |         cycle_start_t = sim_t_total
334 |     elif ev_type == "wave" and sm_state == STATE_IDLE:
335 |         # Randomizar el lado del saludo
336 |         chosen_side = "right" if np.random.uniform() < 0.5 else "left"
337 |         wave_current = wave_right if chosen_side == "right" else wave_left
338 |         wave_current.trigger(sim_t_total)
339 |         wave_start_t = sim_t_total
340 |         sm_state = STATE_WAVING
341 |         print(f"[runtime] EVENT wave recibido -> WAVING ({chosen_side})")
342 |     elif ev_type == "set_skin":
343 |         # Cambio de apariencia - cosmetico, funciona en cualquier estado
344 |         skin_name = ev.get("name", "")
345 |         if apply_skin(model, skin_name):
346 |             print(f"[runtime] EVENT set_skin -> {skin_name}")
347 |         else:
348 |             print(f"[runtime] set_skin desconocido: {skin_name}")
349 |     elif ev_type == "cam":
350 |         # D-pad de la camara: izq/der giran, arriba/abajo cambian altura.
351 |         d = ev.get("dir", "")
352 |         if d == "left":
353 |             stream_cam.azimuth = (stream_cam.azimuth - CAM_AZ_STEP) % 360
354 |         elif d == "right":
355 |             stream_cam.azimuth = (stream_cam.azimuth + CAM_AZ_STEP) % 360
356 |         elif d == "up":
357 |             # subir la camara = elevation mas negativa (clamp a CAM_EL_HIGH)
358 |             stream_cam.elevation = max(CAM_EL_HIGH, stream_cam.elevation - CAM_EL_STEP)
359 |         elif d == "down":
360 |             # bajar = elevation menos negativa, pero no por debajo de CAM_EL_LOW
361 |             stream_cam.elevation = min(CAM_EL_LOW, stream_cam.elevation + CAM_EL_STEP)
362 |         print(f"[runtime] cam {d} -> az={stream_cam.azimuth:.0f} el={stream_cam.elevation:.0f}")
363 | except queue.Empty:
364 |     pass
365 |
366 | # --- 2. Actualizar red mocap target desde slider web ---
367 | try:
368 |     td = target_q.queue[0] # peek
369 |     red_target_world = np.array([float(td["x"]), float(td["y"]), float(td["z"])])
370 | except (IndexError, queue.Empty):
371 |     pass
372 | data.mocap_pos[target_mocap_idx] = red_target_world
373 |
374 | # --- 3. Decidir target del head segun estado ---
375 | ball_pos = data.qpos[arm_env._ball_qpos_adr:arm_env._ball_qpos_adr+3].copy()
376 | # Solo grab cycle (SPAWN_AND_FALL, HELD, THROWN, FOLLOW) usa la bola amarilla
377 | # como target. WAVING e IDLE siguen la bola roja (slider del usuario).
378 | if sm_state in (STATE_SPAWN_AND_FALL, STATE_HELD, STATE_THROWN, STATE_FOLLOW):
379 |     head_target = ball_pos.copy()
380 | else:
381 |     head_target = red_target_world
382 |
383 | # --- 4. Head action ---
384 | head_obs = compute_head_obs(head_target, prev_head_action)
385 | head_obs_np = head_obs[None, :].astype(np.float32)
386 | head_action, _ = head_model.predict(head_obs_np, deterministic=True)
387 | head_action = head_action[0]
388 | prev_head_action = head_action.astype(np.float32).copy()
389 | head_ctrl = head_action_to_ctrl(head_action)
390 |
391 | # --- 5. Arm action ---
392 | if sm_state in (STATE_SPAWN_AND_FALL, STATE_HELD, STATE_THROWN, STATE_FOLLOW):
393 |     # Activar policy del brazo
394 |     arm_obs = arm_env._get_obs()
395 |     arm_obs_norm = arm_vn.normalize_obs(arm_obs[None, :])
396 |     arm_action_raw, _ = arm_model.predict(arm_obs_norm, deterministic=True)
397 |     arm_action = arm_action_raw[0]
398 |     arm_ctrl = arm_env._action_to_arm_ctrl(arm_action)
399 | else:
400 |     # IDLE o WAVING: arm policy desactivado
401 |     arm_ctrl = None
402 |     arm_action = np.zeros(5, dtype=np.float32)
403 |
404 | # --- 5b. Autofoco - calcular theta_to_target y update del state machine ---
405 | head_mat_now = data.xmat[head_body].reshape(3, 3)
406 | forward_world = head_mat_now @ HEAD_FORWARD_LOCAL
407 | lens_origin_now = data.site_xpos[lens_site]
408 | delta = head_target - lens_origin_now
409 | d_norm = float(np.linalg.norm(delta))
410 | if d_norm > 1e-9:
411 |     cos_theta = float(np.clip(np.dot(forward_world, delta / d_norm), -1.0, 1.0))
412 |     theta_to_target = float(np.arccos(cos_theta))
413 | else:

```

```

414 |         theta_to_target = 0.0
415 |     # FSM del foco
416 |     if not focus_triggered:
417 |         # Trigger cuando el head se asienta cerca del target
418 |         if theta_to_target < FOCUS_THRESHOLD:
419 |             focus_settling_counter += 1
420 |             if focus_settling_counter >= FOCUS_SETTLING_STEPS:
421 |                 focus_triggered = True
422 |                 focus_anim_step = 0
423 |                 focus_n_cycles = int(np.random.randint(*FOCUS_N_CYCLES_RANGE))
424 |                 focus_amplitude = float(np.random.uniform(*FOCUS_AMPLITUDE_RANGE))
425 |                 focus_resting = float(np.random.uniform(*FOCUS_RESTING_RANGE)) * LENS_MAX
426 |                 print(f"[autofoco] TRIGGER (theta={np.rad2deg(theta_to_target):.1f}° "
427 |                       f"cycles={focus_n_cycles} amp={focus_amplitude:.2f})")
428 |             else:
429 |                 focus_settling_counter = 0
430 |         else:
431 |             # Re-arm si el target se aleja sostenidamente
432 |             if theta_to_target > FOCUS_THRESHOLD * FOCUS_REARM_THRESHOLD_MULT:
433 |                 focus_lost_steps += 1
434 |                 if focus_lost_steps >= FOCUS_REARM_LOST_STEPS:
435 |                     focus_triggered = False
436 |                     focus_anim_step = -1
437 |                     focus_lost_steps = 0
438 |                     focus_settling_counter = 0
439 |                     print(f"[autofoco] REARM (theta={np.rad2deg(theta_to_target):.1f}°)")
440 |                 else:
441 |                     focus_lost_steps = 0
442 |                     if focus_anim_step >= 0:
443 |                         focus_anim_step += 1
444 |             # Computar lens ctrl segun fase de la animacion
445 |             if not focus_triggered or focus_anim_step < 0:
446 |                 lens_ctrl = 0.0
447 |             elif focus_anim_step >= FOCUS_ANIM_STEPS:
448 |                 lens_ctrl = focus_resting
449 |             else:
450 |                 phase = focus_anim_step / FOCUS_ANIM_STEPS
451 |                 peak = focus_amplitude * LENS_MAX
452 |                 lens_ctrl = peak * 0.5 * (1.0 - np.cos(2.0 * focus_n_cycles * np.pi * phase))
453 |
454 |     # --- 6. Construir full_ctrl ---
455 |     full_ctrl = np.zeros(model.nu, dtype=np.float64)
456 |     for aid in range(model.nu):
457 |         lo, hi = model.actuator_ctrlrange[aid]
458 |         full_ctrl[aid] = 0.0 if (lo <= 0.0 <= hi) else 0.5 * (lo + hi)
459 |     # head ctrl
460 |     for i, aid in enumerate(head_act_ids):
461 |         full_ctrl[aid] = head_ctrl[i]
462 |     # lens ctrl (autofoco)
463 |     full_ctrl[lens_act_id] = lens_ctrl
464 |     # arm ctrl si aplica
465 |     if arm_ctrl is not None:
466 |         for i, aid in enumerate(arm_env._arm_act_ids):
467 |             full_ctrl[aid] = arm_ctrl[i]
468 |     # wave ctrl override (aditivo) - escribe sobre 4 actuadores del brazo elegido
469 |     if sm_state == STATE_WAVING and wave_current is not None:
470 |         wave_ctrl_dict = wave_current.get_ctrl(sim_t_total)
471 |         if wave_ctrl_dict:
472 |             for act_name, val in wave_ctrl_dict.items():
473 |                 aid = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_ACTUATOR, act_name)
474 |                 if aid >= 0:
475 |                     full_ctrl[aid] = val
476 |
477 |     # --- 7. Gravedad efectiva sobre la bola (cancel parcial durante THROWN/FOLLOW) ---
478 |     if sm_state in (STATE_THROWN, STATE_FOLLOW):
479 |         # Aplicar fuerza hacia arriba sobre la bola que cancela ~80% del peso
480 |         ball_mass = model.body_mass[arm_env._ball_body_id]
481 |         grav = model.opt.gravity[2]
482 |         cancel_force_z = -GRAVITY_CANCEL_FACTOR * ball_mass * grav # contrario a peso
483 |         data.xfrc_applied[arm_env._ball_body_id, :] = 0.0
484 |         data.xfrc_applied[arm_env._ball_body_id, 2] = cancel_force_z
485 |     else:
486 |         data.xfrc_applied[arm_env._ball_body_id, :] = 0.0
487 |
488 |     # --- 8. Step ---
489 |     data.ctrl[:] = full_ctrl
490 |     for _ in range(FRAME_SKIP):
491 |         mujoco.mj_step(model, data)
492 |     sim_t_total += sim_dt
493 |
494 |     # En IDLE, pinear la bola al hidden pos (la gravedad la haria caer al infinito)
495 |     if sm_state == STATE_IDLE:
496 |         data.qpos[arm_env._ball_qpos_adr:arm_env._ball_qpos_adr+3] = yellow_hidden_pos
497 |         data.qvel[arm_env._ball_dof_adr:arm_env._ball_dof_adr+6] = 0.0
498 |         mujoco.mj_forward(model, data)

```

```

499 |
500 | # En HELD, manual override del qpos de la bola (sigue la palma)
501 | from rl.envs.grab_env import STATE_FALLING as EnvFalling
502 | if sm_state == STATE_HELD or arm_env.state == "HELD":
503 |     anchor_world = arm_env._ee_pos_world().copy()
504 |     data.qpos[arm_env._ball_qpos_adr:arm_env._ball_qpos_adr+3] = anchor_world
505 |     ee_v = arm_env._ee_linvel_world()
506 |     data.qvel[arm_env._ball_dof_adr:arm_env._ball_dof_adr+3] = ee_v
507 |     data.qvel[arm_env._ball_dof_adr+3:arm_env._ball_dof_adr+6] = 0.0
508 |     mujoco.mj_forward(model, data)
509 |
510 | # --- 9. Transiciones del state machine externo ---
511 | cycle_dt = sim_t_total - cycle_start_t
512 | if sm_state == STATE_SPAWN_AND_FALL:
513 |     # Detectar grab por la condicion del env
514 |     if arm_env.detect_grab() and not arm_env._grab_bonus_consumed:
515 |         arm_env.state = "HELD"
516 |         arm_env.t_grab = sim_t_total - cycle_start_t
517 |         arm_env.was_held = True
518 |         arm_env._grab_bonus_consumed = True
519 |         sm_state = STATE_HELD
520 |         print(f"[runtime] GRAB! @ t={cycle_dt:.2f}s -> HELD")
521 |     # Timeout de catch
522 |     elif cycle_dt > 4.0:
523 |         print(f"[runtime] TIMEOUT catch -> IDLE")
524 |         hide_yellow_ball()
525 |         sm_state = STATE_IDLE
526 |     # Si la bola tocó piso sin grab
527 |     elif ball_pos[2] < 0.05:
528 |         print(f"[runtime] BOLA AL PISO sin grab -> IDLE")
529 |         hide_yellow_ball()
530 |         sm_state = STATE_IDLE
531 | elif sm_state == STATE_HELD:
532 |     # Auto-release tras THROW_HELD_MAX_S. SIN boost - la velocidad
533 |     # sale 100% de la policy. Si la policy no swingueo, la bola cae cerca,
534 |     # y eso es un problema de TRAINING, no del runtime.
535 |     if arm_env.t_grab is not None and (cycle_dt - arm_env.t_grab) > THROW_HELD_MAX_S:
536 |         arm_env.state = "THROWN"
537 |         ee_v = arm_env._ee_linvel_world().copy()
538 |         arm_env._release_ee_velocity = ee_v
539 |         arm_env.t_release = cycle_dt
540 |         # La bola hereda la velocidad del EE (ya seteada via manual override cada step
541 |         # en HELD). NO la modificamos aca - pure policy output.
542 |         sm_state = STATE_THROWN
543 |         follow_start_t = sim_t_total
544 |         policy_forward_v = -float(ee_v[1])
545 |         print(f"[runtime] RELEASE @ t={cycle_dt:.2f}s -> THROWN "
546 |               f"(v_fwd={policy_forward_v:.2f} m/s, v_up={ee_v[2]:.2f} m/s, "
547 |               f"|v|={np.linalg.norm(ee_v):.2f} m/s)")
548 |     elif sm_state == STATE_THROWN:
549 |         # Activar fase FOLLOW (head sigue bola por ~3s)
550 |         if follow_start_t is not None and (sim_t_total - follow_start_t) > 0.2:
551 |             sm_state = STATE_FOLLOW
552 |             print(f"[runtime] -> FOLLOW")
553 |     elif sm_state == STATE_FOLLOW:
554 |         # 3s de follow, despues volver a IDLE
555 |         if follow_start_t is not None and (sim_t_total - follow_start_t) > FOLLOW_DURATION_S:
556 |             print(f"[runtime] FOLLOW done -> IDLE (head vuelve a red ball)")
557 |             hide_yellow_ball()
558 |             sm_state = STATE_IDLE
559 |             target_world_prev = red_target_world.copy() # reset para no romper tvel
560 |             # 0 si la bola se va muy lejos
561 |             elif ball_pos[2] < 0.02 or np.linalg.norm(ball_pos) > 5.0:
562 |                 print(f"[runtime] bola fuera de rango -> IDLE")
563 |                 hide_yellow_ball()
564 |                 sm_state = STATE_IDLE
565 |                 target_world_prev = red_target_world.copy()
566 | elif sm_state == STATE_WAVING:
567 |     # Wave dura DURATION_S (6s en v2) - el WaveAnimation se apaga solo.
568 |     if wave_current is not None and not wave_current.active:
569 |         print(f"[runtime] WAVING done -> IDLE")
570 |         wave_current = None
571 |         wave_start_t = None
572 |         sm_state = STATE_IDLE
573 |
574 | target_world_prev = head_target.copy()
575 |
576 | # --- 10. Render frame para stream MJPEG ---
577 | frame_counter += 1
578 | if frame_counter >= frame_every_n:
579 |     frame_counter = 0
580 |     try:
581 |         stream_renderer.update_scene(data, camera=stream_cam)
582 |         rgb = stream_renderer.render()
583 |         buf = io.BytesIO()

```

```

584 |         Image.fromarray(rgb).save(buf, format="JPEG", quality=75)
585 |         try:
586 |             frame_q.get_nowait()
587 |         except queue.Empty:
588 |             pass
589 |         frame_q.put_nowait(buf.getvalue())
590 |     except Exception as e:
591 |         pass
592 |
593 |     # --- 11. Telemetria al cliente (10 Hz) ---
594 |     if sim_t_total - last_telemetry_t >= 0.1:
595 |         last_telemetry_t = sim_t_total
596 |         try:
597 |             telemetry_q.get_nowait()
598 |         except queue.Empty:
599 |             pass
600 |         try:
601 |             telemetry_q.put_nowait({
602 |                 "state": sm_state,
603 |                 "ball_z": float(ball_pos[2]),
604 |                 "fps": 50.0,
605 |             })
606 |         except queue.Full:
607 |             pass
608 |
609 |     # Viewer sync
610 |     if viewer_ctx is not None:
611 |         viewer_ctx.sync()
612 |
613 |     # Cadencia 50Hz
614 |     elapsed = time.perf_counter() - loop_start
615 |     sleep_for = sim_dt - elapsed
616 |     if sleep_for > 0:
617 |         time.sleep(sleep_for)
618 |
619 |     # Log periodico
620 |     now = time.perf_counter()
621 |     if now - last_log_t >= 5.0:
622 |         last_log_t = now
623 |         print(f"[runtime] state={sm_state} sim_t={sim_t_total:.1f}s ball_z={ball_pos[2]*100:.1f}cm")
624 |     except KeyboardInterrupt:
625 |         print("\n[runtime] CTRL+C, saliendo")
626 |     finally:
627 |         if viewer_ctx is not None:
628 |             viewer_ctx.close()
629 |
630 |
631 | if __name__ == "__main__":
632 |     main()
633 |

```